

"If a subject with type unconfined_t creates a new object/subject whose the parent object/subject has type of root_t, set the new object's/subject's type to etc_runtime_t."

In the absence of such a rule, the newly created object would simply have the type of its parent object. These are called type transition rules.

To check the number of type transition rules, in our policy, use the *seinfo* tool:

```
[root@vbg-work html]# seinfo
```

Statistics for policy file: /etc/selinux/targeted/policy/policy24

Policy Version & Type: v24 (binary, mls)

Classes:	75	Permissions:	229
Sensitivities:	1	Categories:	1024
Types:	2804	Attributes:	226
Users:	8	Roles:	12
Booleans:	146	Cond. Expr.:	177
Allow:	223680	Neverallow:	0
Auditallow:	39	Dontaudit:	107109
Type_trans:	9675	Type_change:	38
Type_member:	57	Role allow:	19
Rule_trans:	194	Range_trans:	1802
Constraints:	62	Validatetrans:	0
Initial SIDe:	27	Fa_use:	23
Genfscon:	62	Portcon:	403
Netifcon:	0	Nodecon:	0
Permissions:	30	Polcap:	2

Going by the above output, we see that there are 9675 type transition rules in our default policy. To see all of them, simply run the following command:

```
[root@vbg-work html]# search -T
```

The output screen will display all the type transition rules. To see the top few type transition rules, we can use the command given below:

```
[root@vbg-work html]# search -T | head
Found 9770 semantic rules:
type_transition nx_server_t devpts_t : chr_file nx_server_devpts_t;
type_transition user_wine_t fusemount_exec_t : process mount_t;
type_transition staff_execmem_t nslplugin_config_exec_t : process
nslplugin_config_t;
type_transition inatmod_t fusemount_exec_t : process mount_t;
type_transition xend_t fusemount_exec_t : process mount_t;
type_transition sysadm_java_t fusemount_exec_t : process mount_t;
type_transition rictd_modservice_t fail2ban_initro_exec_t : process initro_t;
type_transition init_t a1s_bosserver_exec_t : process a1s_bosserver_t;
type_transition hotplug_t dictd_initro_exec_t : process initro_t;
```

This gives us an opportunity to understand the syntax of a type transition rule, which can be specified as:

```
type_transition [Creating Subject type] [Parent Object/Subject Type] :
```

```
[Class of new Object/Subject] [New Object/Subject Type]
```

This helps us to understand the topmost rule as: If a new object of class *chr_file* (character file) is being created by a process with type *nx_server_t* under a parent object with type *devpts_t*, set the type of the new character file as *nx_server_devpts_t*.

Let us check our policy to see whether a type transition rule exists to change the type of a newly created file under / by the bash process.

As per the above syntax, it should be as follows:

```
type_transition unconfined_t root_t :file etc_runtime_t
[Creating subject - bash] (Parent folder /) (Class of the created object) (New object type)
```

Let us search the policy for this rule. A rudimentary command for the same would be:

```
[root@vbg-work html]# search -T | grep unconfined_t | grep root_t | grep
etc_runtime_t
type_transition unconfined_t root_t :file etc_runtime_t;
```

Therefore, we see that it is this rule that is changing the type of any file being created in the / directory. The above command also shows that this rule will not apply to any directories created under / since it is only applicable on the class "file". Let us check this out:

```
[root@vbg-work html]# mkdir /test
[root@vbg-work html]# ls -ld /test
drwxr-xr-x root.root unconfined_u:object_r:default_t:s0 /test
```

Should the type of the folder /test not be *root_t*? Then why is the type *default_t*? **Hint:** Search for a type transition rule in the SELinux policies to apply to directories under the / directory.

Let us now create a type transition rule in our policy module test so that any files created under the folder /var/www/html/ using the terminal (bash process) will be of the type *lly_t*. The type transition rule will be:

```
type_transition unconfined_t httpd_sys_content_t :file lly_t;
```

Let us add this line to our *test.te* file so that it now looks like (note the change in the version number) what's given below:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
policy_module(test, 1.1.3)

require {
    type unconfined_t;
    type fs_t;
    class file getattr;
    type httpd_sys_content_t; #you will need to add the required type as well
}
```

```
type lly_t;
```